RENTY — Full System Architecture & Business Specification (MVP)

---

1. System Overview

1.1 Purpose

Renty is a sports equipment rental and purchase platform targeting:

- Egyptian local users
- Sports tourists visiting Egypt

The system enables:

- Equipment rental
- Equipment purchase
- Fraud-aware transactions
- Expert verification before and after rentals
- Data-driven supply-demand gap analysis
- Recommendation-based personalization

---

2. Business Model

2.1 Revenue Streams

1. Rental Commission
   The platform takes a percentage from each completed rental.
2. Sales Commission
   A percentage is taken from each purchase transaction.
3. Premium Listings (Future)
   Items can be marked as premium for higher visibility.

2.2 Trust & Guarantee Model

Deposit-Based Guarantee

Each rental includes:

- deposit_amount
- deposit_status (held, returned, forfeited)

Rules:

- Deposit is held during rental.
- Deposit is returned after successful completion.
- Deposit may be forfeited in case of damage or severe delay.

Fraud detection is applied before confirming rentals.

---

3. Roles & Permissions

3.1 Roles

User

- Browse items
- Rent items
- Buy items
- Leave reviews
- Favorite items
- Chat with owners
- Open support tickets

Expert

The Expert is a verified technical inspector assigned by Admin.

Responsibilities:

- Perform item verification:
    - Before rental (before_rent)
    - After return (after_return)
- Update item_verifications:
    - status (approved / rejected)
    - notes
- Confirm item condition
- Flag damaged items

- Recommend deposit forfeiture (all or partial) if necessary

Rules:

- Expert cannot rent or verify their own item.
- Expert actions are logged in item_verifications.
- Expert is assigned by Admin.
- Expert may optionally be linked to inspection points (future expansion).

Admin

- Manage items (suspend/remove)
- Assign experts to verification
- Monitor fraud logs
- View KPIs
- View gap analysis
- Resolve disputes escalated from support
- Activate/deactivate users

Support

- Handle support
- Reply to users
- Escalate disputes to Admin
- Assist with order issues

Support does not:

- Modify financial data
- Change fraud decisions
- Assign experts

Super Admin

- Create Admin accounts
- Create Support accounts
- Create Expert accounts
- Full system access
- Override critical decisions if required

---

4. Core System Modules

4.1 Authentication & Authorization

- JWT access tokens
- Email verification required before renting
- Role-based authorization
- Permission-based control (optional)
- is_active and is_verified must be true for rental

4.2 Items Module

Rules:

- Only item owner can edit item.
- item_status controls availability:
    - available
    - rented
    - under_review
    - suspended
    - removed

An item cannot be rented if:

- status is not available

- the user is the owner

4.3 Rental Flow

Create Rental

Steps:

1. Validate item availability.
2. Prevent self-rental (enforced at DB level).
3. Calculate rental_price from pricing.
4. Set deposit_amount.
5. Call fraud detection.
6. Insert rental with status = pending.
7. Create order (linked to rental).
8. Process payment.
9. Update rental status to confirmed or active.
10. Update item status to rented.

Complete Rental

1. Set actual_return_date.
2. Update rental status to completed.
3. Update deposit_status to returned.
4. Set item status to available.

Overdue Handling

If end_date < now and rental still active:

- status may become overdue
- late_fee may be applied

4.4 Buy Flow

1. Create order.
2. Insert order_items with price_at_time snapshot.

3. Calculate total_amount.
4. Process payment.
5. Update order status:
    - pending
    - paid
    - shipped
    - delivered
    - cancelled

4.5 Payment System (Simulated for MVP)

- Methods: vodafone_cash, fawry, paymob, credit_card, bank_transfer
- Status:
    - pending
    - success
    - failed

Simulation logic:

- Majority success (e.g., 90%)
- On success → order.status = paid
- On failure → order remains pending or failed

No real gateway integration in MVP.

4.6 Delivery System (Simulated)

delivery_tracking records:

- order_id
- status
- location
- updated_at

Status progression:

processing → shipped → in_transit → delivered

4.7 Search System

Every search:

- Returns filtered items.
- Logs into search_logs table.

search_logs is used for:

- Demand measurement
- Gap analysis
- Behavioral analytics

4.8 Recommendation System (RS)

Triggered when:

- User opens home page.

Inputs:

- user_interactions
- search_logs
- rentals
- favorites

Outputs:

- recommendations table
- score (0–1)
- algorithm type
- reason

4.9 Fraud Detection (FD)

Triggered when:

- Rental is initiated.

Inputs:

- User rental history
- Cancellation patterns
- Behavioral signals

Outputs:

- fraud_logs
- risk_score (0–1)
- decision (safe, flagged, blocked)

If blocked:

- Rental cannot proceed.

4.10 Gap Analysis (GAP)

Runs daily via analytics pipeline (n8n).

Inputs:

- search_logs (demand)
- items (supply)

gap_score = demand / supply

Stored in:

- gap_results

Used by:

- Admin dashboard
- Future strategic insights

4.11 KPI System

kpis_snapshots_derived stores:

- active_users
- new_users
- total_items
- active_rentals
- rentals
- revenue
- avg_rental_duration
- fraud_rate
- recommendation_ctr
- gap_score_avg
- avg_order_value
- cancellation_rate
- search_count

Computed daily.

4.12 Support System

Tables:

- support_tickets
- support_messages

Flow:

1. User creates ticket.
2. Support replies via messages.
3. Status transitions:
    - open
    - in_progress
    - resolved

- closed

4.13 Chat System

User-to-user communication.

Tables:

- chats
- chat_messages

Rules:

- Only participants can access chat.
- is_read tracks message state.

4.14 Reviews

Rules:

- Only after completed rental.
- One review per rental.
- rating enforced between 1 and 5.
- Stored in reviews table.

4.15 Notifications

Stored in notifications table.

Fields:

- send_email
- status (pending, sent, failed)
- is_read

Used for:

- Payment confirmation
- Rental updates
- Fraud alerts

- Support replies

Email service reads notifications with send_email = true.

---

5. Data Architecture

Operational Data:

- PostgreSQL

Analytics Data:

- PostgreSQL (logs)
- Azure Data Lake (future training)

Images:

- Cloudinary

---

6. Infrastructure

Frontend:

- Angular (Web)
- Flutter (Mobile)

Backend:

- ASP.NET Core Web API

Database:

- PostgreSQL (Azure Flexible Server/Supabase)

Hosting:

- Azure App Service

Analytics:

- n8n pipeline

---

7. Security Model

- JWT authentication
- Role-based access control
- ENUM-based status enforcement
- CHECK constraints
- Foreign key constraints
- Prevent self-rental trigger
- Deposit system
- Fraud logging

---

8. Scalability Strategy

Initial MVP:

- Single backend instance
- Indexed queries
- Optimized schema

Scaling Plan:

- Add Redis caching
- Add read replicas
- Horizontal scaling via Azure App Service
- Rolling deployment with no downtime

---

9. Data Integrity Rules

- Cannot rent own item.
- Cannot rent unavailable item.
- Must verify email before renting.
- One review per rental.
- All monetary values use decimal.
- Status values enforced via ENUM.

- updated_at auto-updated via trigger.
- All search activity logged.

---

10. MVP Scope Boundaries

Included:

- Renting
- Buying
- Deposit logic
- Fraud logging
- Recommendations
- Gap analysis
- Support system
- Chat
- Notifications
- Admin dashboard

Excluded:

- Real payment gateway integration
- Real delivery API integration
- Microservices architecture
- Multi-country deployment